

INSTITUTO FEDERAL DE MINAS GERAIS (IFMG)
CAMPUS BAMBUÍ
BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

Disciplina: BiSuEEA.512 – Sistemas Embarcados
Professor: Williams L. Nicomedes

Henrique Augusto G. Fernandes

TRABALHO Nº2 – CONTADOR MOD 8 ENCAPSULADO

SUMÁRIO

1	INTRODUÇÃO	3
2	DESENVOLVIMENTO	4
2.1	Fundamentos Teóricos	4
2.1.1	<i>Conceito de Encapsulamento</i>	4
2.1.2	<i>Instanciação de Componentes em VHDL</i>	4
2.1.3	<i>Arquitetura do Contador Encapsulado</i>	4
2.2	Implementação do Contador Encapsulado	5
2.2.1	<i>Código VHDL do Flip-Flop JK</i>	5
2.2.2	<i>Código VHDL do Contador Encapsulado</i>	5
2.2.3	<i>Estrutura do Código</i>	5
2.3	Implementação no Quartus II	6
2.3.1	<i>Configuração do Projeto</i>	6
2.3.2	<i>Diagrama Esquemático</i>	7
2.4	Simulação e Validação	7
2.4.1	<i>Configuração das Formas de Onda</i>	7
2.4.2	<i>Resultados da Simulação</i>	7
2.4.3	<i>Análise dos Resultados</i>	7
3	CONCLUSÃO	9

1 INTRODUÇÃO

Este documento apresenta a resolução do Trabalho n°2 da disciplina de Sistemas Embarcados, referente ao 2° semestre de 2025. O objetivo consiste em implementar um contador MOD 8 **encapsulado** em um único bloco VHDL, dando continuidade ao Trabalho n°1 onde o contador foi construído através da interconexão de três blocos de flip-flops no esquemático.

Conforme apresentado nas Notas de Aula 21, o **encapsulamento** é uma técnica que permite criar um componente único contendo internamente toda a lógica do circuito. Ao invés de expor os três flip-flops JK no esquemático, toda a interconexão é realizada dentro do código VHDL, resultando em um bloco com interface simplificada:

- **Entrada única:** sinal de clock;
- **Saída única:** vetor de 3 bits representando a contagem.

O trabalho está estruturado nas seguintes etapas:

- a) criação do código VHDL do contador encapsulado utilizando instanciação de componentes;
- b) construção do esquemático simplificado com o bloco único;
- c) verificação do funcionamento através de simulação.

O desenvolvimento foi realizado no software Intel Quartus II, utilizando a técnica de **instanciação de componentes** para reutilizar o código do flip-flop JK desenvolvido no Trabalho n°1.

2 DESENVOLVIMENTO

2.1 Fundamentos Teóricos

2.1.1 Conceito de Encapsulamento

O encapsulamento em VHDL consiste em criar uma entidade de nível superior que instancia e interconecta componentes internamente, expondo apenas uma interface simplificada ao usuário. A Tabela 1 apresenta as diferenças entre as abordagens com e sem encapsulamento.

Tabela 1 – Comparação entre abordagens de implementação do contador MOD 8

Aspecto	Trabalho 1	Trabalho 2
Esquemático	3 blocos	1 bloco
Conexões	Manuais	Internas
Complexidade	Maior	Menor
Reusabilidade	Baixa	Alta
Abstração	Baixa	Alta

Fonte: Elaborado pelo autor, 2026.

2.1.2 Instanciação de Componentes em VHDL

Para utilizar um bloco VHDL dentro de outro, são necessários três passos:

- Declaração do componente:** dentro da arquitetura, declara-se a interface do componente a ser instanciado, utilizando a palavra-chave `component`;
- Declaração de sinais internos:** criam-se sinais para realizar as interconexões internas entre os componentes;
- Instanciação com port map:** cria-se uma ou mais instâncias do componente, mapeando os sinais internos aos pinos através do comando `port map`.

2.1.3 Arquitetura do Contador Encapsulado

O contador encapsulado contém internamente três instâncias do flip-flop JK (FF0, FF1, FF2), conectados em cascata. A estrutura interna do bloco Contador_MOD8 é organizada da seguinte forma:

- **FF0** (LSB): recebe o clock externo; saída Q0 conecta ao clock de FF1;
- **FF1**: recebe Q0 como clock; saída Q1 conecta ao clock de FF2;

- **FF2 (MSB):** recebe Q1 como clock; saída Q2;
- **Entradas J e K:** conectadas a Vcc para modo toggle;
- **Saída:** vetor contador formado por Q2, Q1 e Q0.

2.2 Implementação do Contador Encapsulado

2.2.1 Código VHDL do Flip-Flop JK

O Quadro 1 apresenta o código do flip-flop JK, desenvolvido no Trabalho nº1 e reutilizado como componente neste trabalho.

Quadro 1 – Código VHDL do flip-flop tipo JK

```
library ieee;
use ieee.std_logic_1164.all;

entity BlocoFF_JK is
port(
J   : in  std_logic;
K   : in  std_logic;
clk : in  std_logic;
Q   : out std_logic
);
end BlocoFF_JK;

architecture funcionamento of BlocoFF_JK is
signal Qstate : std_logic := '0';
begin
process(J, K, clk)
begin
if falling_edge(clk) then
if J = '1' and K = '1' then
Qstate <= not(Qstate);
elsif J = '1' and K = '0' then
Qstate <= '1';
elsif J = '0' and K = '1' then
Qstate <= '0';
end if;
end if;
end process;
Q <= Qstate;
end funcionamento;
```

Fonte: Elaborado pelo autor, 2026.

2.2.2 Código VHDL do Contador Encapsulado

O Quadro 2 apresenta o código VHDL do contador MOD 8 encapsulado.

2.2.3 Estrutura do Código

O código do contador encapsulado está organizado conforme a estrutura padrão:

- Entidade:** interface simplificada com clock e contador;
- Declaração de componente:** referência ao bloco BlocoFF_JK;

Quadro 2 – Código VHDL do contador MOD 8 encapsulado

```

library ieee;
use ieee.std_logic_1164.all;

entity Contador_MOD8 is
port(
clock   : in  std_logic;
contador: out std_logic_vector(2 downto 0)
);
end Contador_MOD8;

architecture estrutural of Contador_MOD8 is
component BlocoFF_JK is
port(
J      : in  std_logic;
K      : in  std_logic;
clk    : in  std_logic;
Q      : out std_logic
);
end component;

signal Q0_interno : std_logic;
signal Q1_interno : std_logic;
signal Q2_interno : std_logic;
signal Vcc        : std_logic := '1';

begin
FF0: BlocoFF_JK port map(
J => Vcc, K => Vcc, clk => clock, Q => Q0_interno);

FF1: BlocoFF_JK port map(
J => Vcc, K => Vcc, clk => Q0_interno, Q => Q1_interno);

FF2: BlocoFF_JK port map(
J => Vcc, K => Vcc, clk => Q1_interno, Q => Q2_interno);

contador(0) <= Q0_interno;
contador(1) <= Q1_interno;
contador(2) <= Q2_interno;
end estrutural;

```

Fonte: Elaborado pelo autor, 2026.

- c) **Sinais internos:** Q0_interno, Q1_interno, Q2_interno e Vcc;
- d) **Instanciação:** três instâncias do flip-flop JK com port map.

2.3 Implementação no Quartus II

2.3.1 Configuração do Projeto

O projeto requer dois arquivos VHDL:

- a) BlocoFF_JK.vhd – código do flip-flop JK;
- b) Contador_MOD8.vhd – código do contador encapsulado.

Para cada arquivo:

- a) **Processing** → **Analyze Current File**;
- b) **File** → **Create/Update** → **Create Symbol Files for Current File**.

2.3.2 Diagrama Esquemático

A Figura 1 apresenta o diagrama esquemático simplificado, contendo apenas o bloco encapsulado Contador_MOD8.

Figura 1 – Diagrama esquemático do contador MOD 8 encapsulado



Fonte: Elaborado pelo autor, 2026.

Conforme observado na Figura 1, o esquemático apresenta:

- **Entrada:** CLK_IN conectada ao pino clock do bloco;
- **Saída:** CONT_OUT conectada ao pino contador do bloco;
- **Bloco único:** Contador_MOD8.

2.4 Simulação e Validação

2.4.1 Configuração das Formas de Onda

Para validar o funcionamento do contador encapsulado, foi criado um arquivo de simulação com as seguintes configurações:

- **End Time:** 1,6 μ s;
- **Grid Size:** 50 ns;
- **CLK_IN:** clock com período de 100 ns.

2.4.2 Resultados da Simulação

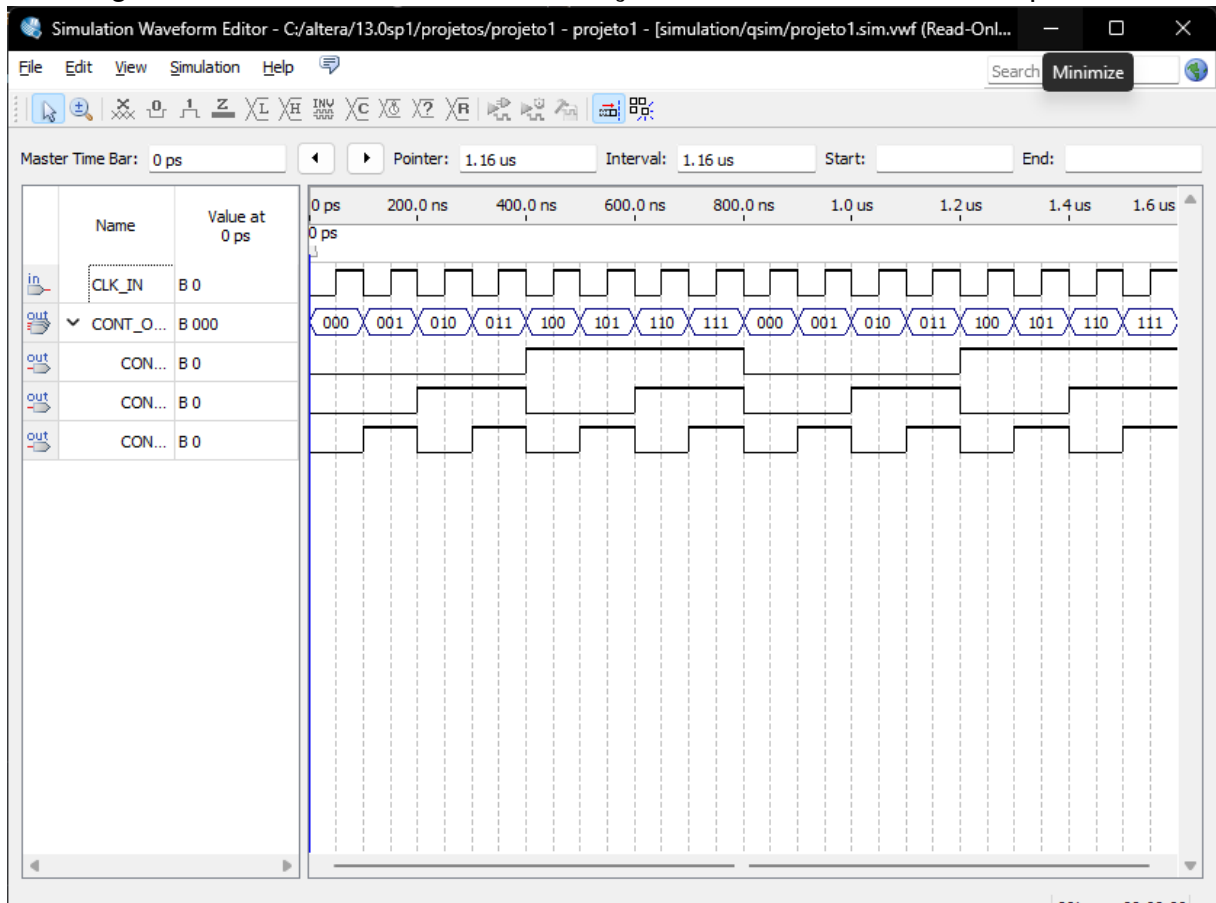
A Figura 2 apresenta as formas de onda resultantes da simulação funcional.

2.4.3 Análise dos Resultados

Analisando as formas de onda apresentadas na Figura 2, verifica-se:

- a) Vetor CONT_OUT: exibe a sequência de contagem binária de 000 a 111;
- b) CONT_OUT[0] (LSB): alterna a cada pulso de clock;
- c) CONT_OUT[1]: alterna a cada dois pulsos;
- d) CONT_OUT[2] (MSB): alterna a cada quatro pulsos.

Figura 2 – Formas de onda da simulação do contador MOD 8 encapsulado



Fonte: Elaborado pelo autor, 2026.

A Tabela 2 apresenta a correspondência entre os pulsos de clock e os estados do contador.

Tabela 2 – Verificação dos estados do contador MOD 8 encapsulado

Pulso	CONT_OUT	Decimal
0	000	0
1	001	1
2	010	2
3	011	3
4	100	4
5	101	5
6	110	6
7	111	7
8	000	0 (reinicia)

Fonte: Elaborado pelo autor, 2026.

Os resultados obtidos correspondem integralmente ao comportamento esperado.

3 CONCLUSÃO

Este trabalho apresentou o projeto e implementação de um contador MOD 8 encapsulado em um único bloco VHDL, dando continuidade ao Trabalho nº1.

A técnica de encapsulamento proporcionou as seguintes vantagens:

- Simplificação do esquemático: redução de três blocos para apenas um;
- Ocultação da complexidade: as interconexões internas ficam encapsuladas no código VHDL;
- Reusabilidade: o bloco Contador_MOD8 pode ser reutilizado em outros projetos;
- Interface limpa: entrada única (clock) e saída única (vetor de contagem).

A simulação funcional no Quartus II validou completamente o comportamento do contador encapsulado, demonstrando contagem correta de 0 a 7 em sequência binária crescente e retorno automático ao estado inicial após o estado 7.